幻尔科技

树莓派版本开发教程

V1.0

Hiwonder 官方网站

版本号	修改日期	修改摘要
V1.0	20230923	初次发布

目录

1.准备工作 4
1.1 接线说明 4
1.2 环境配置 4
2.案例开发 5
2.1 案例 1 总线舵机信息读取 5
2.1.1 运行程序 5
2.1.2 实现效果 5
2.1.3 案例程序简要分析 6
2.2 案例 2 总线舵机 ID 设置 7
2.2.1 运行程序 7
2.2.2 实现效果 8
2.2.3 案例程序简要分析 9
2.3 案例 3 控制总线舵机转动 13
2.3.1 运行程序 13
2.3.2 实现效果 13
2.3.3 案例程序简要分析 13
2.4 案例 4 调节总线舵机速度 14
2.4.1 运行程序 14
2.4.2 实现效果 14
2.4.3 案例程序简要分析 15

2.5 案例 5 示教记录操作 16

2.5.1 运行程序 16

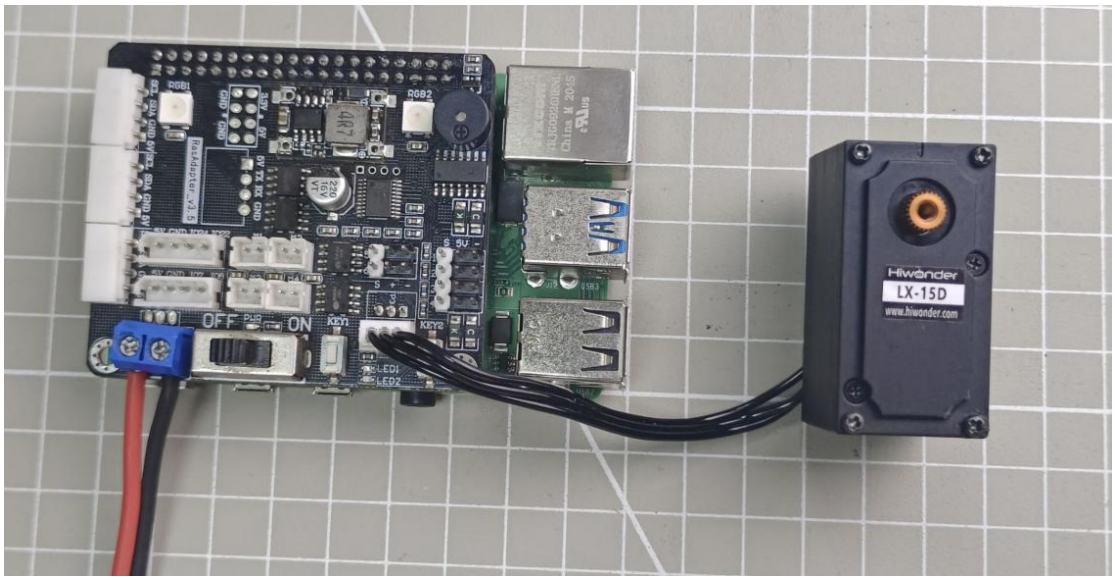
2.5.2 实现效果 17

2.5.3 案例程序简要分析 17

1.准备工作

1.1 接线说明

本节示例使用的是树莓派主板和树莓派扩展板，通过 7.4V 2200mAh 锂电池来供电。将总线舵机连接至扩展板任意一个总线接口。



注意：

① 如使用我司锂电池，连接锂电池对接线请以红接＋，黑接-接到 DC 接口。

② 如对接线未连接锂电池，请勿与电池对接线直接对接，避免正负极发生短路从而造成短路。

1.2 环境配置

在电脑端安装 NoMachine，软件包位于“**2 软件工具->远程桌面连接工具**”下。关于 NoMachine 的详细使用，可在对应目录下进行学习。

将程序和库文件 SDK，拖动到树莓派系统镜像内，这里以放置在桌面为例进行。**注意：库文件要同程序放在同一目录下。**

打开命令行终端，输入添加执行权限的指令：

“sudo chmod a+x Raspberry_BusServo_demo/”。

```
ubuntu@ubuntu:~/Desktop$ sudo chmod a+x Raspberry_BusServo_demo/
ubuntu@ubuntu:~/Desktop$
```

2.案例开发

2.1 案例 1 总线舵机信息读取

本案例通过终端窗口显示出总线舵机 ID、位置、温度等相关信息。

```
id:5
pos:886
dev:38
angle_range:(0, 1000)
voltage_range:(4500, 14000)
temperature_warn:85
temperature:34
vin:11905
lock:0
```

2.1.1 运行程序

(1) 打开终端，输入切换到程序所在目录的指令：

“cd Raspberry_BusServo_demo/”，按下回车。

```
ubuntu@ubuntu:~/Desktop$ cd Raspberry_BusServo_demo/
ubuntu@ubuntu:~/Desktop/Raspberry_BusServo_demo$
```

(2) 输入执行本案例程序的指令：

“python3 BusServo_ReadStatus.py”。

```
hiwonder@hiwonder:~/Desktop/serial_servo$ python3 serial_servo_status.py
```

2.1.2 实现效果

程序运行后，终端打印画面会滚屏打印舵机各项状态信息。

```
Pulse:  498
Temp:   56
Vin:    11524
id:    1
```

各项信息的具体含义如下：

① Pulse：当前舵机转动的位置，这里为 498。关于位置和角度的换算可参考“1.使用教程->配套手册->总线舵机使用说明”。

② Temp：当前舵机温度，这里为 56 摄氏度。

③ Vin：当前舵机电压，这里为 11.524V。

④ id：当前舵机 ID，这里为 1。

2.1.3 案例程序简要分析

- **导入必要的模块**

```
import time
import Board
```

首先，导入 Board 和 time 模块。通过导入该模块，我们可以使用其中定义的函数和类来控制舵机。

- **获取舵机状态信息并打印**

```
    Pulse = Board.getBusServoPulse(servo_id) # 获取舵机的位置信息
    Temp = Board.getBusServoTemp(servo_id) # 获取舵机的温度信息
    Vin = Board.getBusServoVin(servo_id) # 获取舵机的电压信息
```

进入循环函数后，通过调用 Board 对象的方法，获取舵机的各种状态信息。这些状态信息包括舵机位置、温度、电压以及舵机上电状态。

在获取后，可将上面的这些参数打印出来。

```
        print('Pulse: {}\nTemp:  {}\nVin:   {}\nid:   {}\n'.format(Pulse,
Temp, Vin, servo_id))
```

将获取到的舵机状态信息打印出来。

2.2 案例 2 总线舵机 ID 设置

2.2.1 运行程序

(1) 打开终端，输入切换到程序所在目录的指令：

“**cd Raspberry_BusServo_demo/**”，按下回车。

```
ubuntu@ubuntu:~/Desktop$ cd Raspberry_BusServo_demo/
ubuntu@ubuntu:~/Desktop/Raspberry_BusServo_demo$
```

(2) 输入执行本案例程序的指令：

“**sudo python3 set_BusServo_status.py**”。

```
ubuntu@ubuntu:~/Desktop/Raspberry_BusServo_demo$ sudo python3 set_BusServo_status.py
```

2.2.2 实现效果

```
---------------------------
1 id
2 dev
3 save_dev
4 help
5 voltage_range
6 temperature_warn
7 lock
8 exit
---------------------------
*******current status**********
id:1
dev:-56
angle_range:(0, 1000)
voltage_range:(4500, 14000)
temperature_warn:85
lock:1
*******************************
input mode:
```

在提示栏目中：

1 id　设置 id

2 dev 设置偏差

3 save_dev　保存偏差

4 help　　操作提示

5 voltage_range　电压范围设置

6 temperature_warn 温度警报设置的值

7 lock　　输入 1 是上电，输入 0 是掉电

8 exit　　退出

在“input mode”里输入 1 后，输入要设置的 ID 回车即可。（注意 ID 设置范围应为 0-253）。

2.2.3 案例程序简要分析

- **导入必要的模块**

```
import time
import Board
```

首先，导入 Board 和 time 模块。通过导入该模块，我们可以使用其中定义的函数和类来控制舵机。

- **获取舵机信息并打印**

```
def get_status():
    servo_id = Board.getBusServoID()
    dev = Board.getBusServoDeviation(servo_id)
    if dev > 125:
        dev = -(0xff - (dev - 1))
    angle_range = Board.getBusServoAngleLimit(servo_id)
    vin_range = Board.getBusServoVin(servo_id)
    temperature_warn = Board.getBusServoTempLimit(servo_id)
    load_state = Board.getBusServoLoadStatus(servo_id)
    print('*******current status**********')
    print('id:%s'%(str(servo_id).ljust(3)))
    print('dev:%s'%(str(dev).ljust(4)))
    print('angle_range:%s'%(str(angle_range).ljust(4)))
    print('voltage_range:%s'%(str(vin_range).ljust(5)))
    print('temperature_warn:%s'%(str(temperature_warn).ljust(4)))
    print('lock:%s'%(str(load_state).ljust(4)))
    print('******************************')
```

将获取到的舵机状态信息打印出来。每条打印语句使用字符串格式化和 ljust()方法，以对齐输出结果。

- **进入主循环函数**

```
    while True:
        try:
            get_status()
            mode = int(input('input mode:'))
            if mode == 1:
                oldid = int(input('current id:'))
                newid = int(input('new id(0-253):'))
                Board.setBusServoID(oldid, newid)
            elif mode == 2:
                servo_id = int(input('servo id:'))
                dev = int(input('deviation(-125~125):'))
                if dev < 0:
                    dev = 0xff + dev + 1
                Board.setBusServoDeviation(servo_id, dev)
            elif mode == 3:
                servo_id = int(input('servo id:'))
                Board.saveBusServoDeviation(servo_id)
            # elif mode == 4:
            #     servo_id = int(input('servo id:'))
            #     min_pos = int(input('min pos(0-1000):'))
            #     max_pos = int(input('max pos(0-1000):'))
            #     Board.setBusServoAngleLimit(servo_id, min_pos,
max_pos)
            elif mode == 5:
                servo_id = int(input('servo id:'))
```

```
            min_vin = int(input('min vin(4500-14000):'))
            max_vin = int(input('max vin(4500-14000):'))
            Board.setBusServoVinLimit(servo_id, min_vin, max_vin)
        elif mode == 6:
            servo_id = int(input('servo id:'))
            max_temp = int(input('temperature(0-85):'))
            Board.setBusServoMaxTemp(servo_id, max_temp)
        elif mode == 7:
            servo_id = int(input('servo id:'))
            status = int(input('status 0 or 1:'))
            Board.unloadBusServo(servo_id, status)
        elif mode == 4:
            print('''
--------------------------
1 id
2 dev
3 save_dev
4 help
5 voltage_range
6 temperature_warn
7 lock
8 exit
--------------------------''')
        elif mode == 8:
            break
        else:
            print('error mode')
    except KeyboardInterrupt:
```

```
        break
```

在主循环中，程序首先调用 get_status()函数打印舵机的当前状态。然后，通过输入模式选择用户想要执行的操作。根据不同的模式，程序执行相应的代码块来实现舵机的设置和控制。

- **打印菜单选项**

```
print('''
--------------------------
1 id
2 dev
3 save_dev
4 help
5 voltage_range
6 temperature_warn
7 lock
8 exit
--------------------------''')
```

用于打印菜单选项，提供了不同的操作模式供用户选择。

- **设置中断标志位**

```
    except KeyboardInterrupt:
        break
```

最后设置按键，当终端界面打印时，按下 Ctrl+C，捕获 KeyboardInterrupt 异常，跳出循环，结束程序的执行。

2.3 案例 3 控制总线舵机转动

2.3.1 运行程序

(1) 打开终端，输入切换到程序所在目录的指令：

“cd Raspberry_BusServo_demo/”，按下回车。

```
ubuntu@ubuntu:~/Desktop$ cd Raspberry_BusServo_demo/
ubuntu@ubuntu:~/Desktop/Raspberry_BusServo_demo$
```

(2) 输入执行本案例程序的指令：

“sudo python3 BusServo_Move.py”。点击转动

```
ubuntu@ubuntu:~/Desktop/Raspberry_BusServo_demo$ sudo python3 BusServo_Move.py
```

2.3.2 实现效果

程序运行后，舵机会从位置 200-800 以 1 秒为间隔来回转动。

2.3.3 案例程序简要分析

- **导入必要的模块**

```
import time
import Board
```

首先，导入 Board 和 time 模块。通过导入该模块，我们可以使用其中定义的函数和类来控制舵机。

- **进入主循环函数**

```
while True:
    # 参数：参数 1：舵机 id；参数 2：位置；参数 3：运行时间
    # 舵机的转动范围 0-240 度，对应的脉宽为 0-1000,即参数 2 的范围为 0-1000

    Board.setBusServoPulse(1, 800, 1000) # 1 号舵机转到 800 位置，用时
1000ms
```

```
        time.sleep(0.5) # 延时 0.5s
        Board.setBusServoPulse(1, 200, 1000) # 1 号舵机转到 200 位置，用时
1000ms
        time.sleep(0.5) # 延时 0.5s
```

在主循环中，程序通过调用 Board.setBusServoPulse()方法来设置舵机的位置和运行时间。首先，将舵机 ID 设置为 1，位置设置为 800，运行时间设置为 1000ms，然后调用 Board.setBusServoPulse()方法来控制舵机运动到指定位置。

接着，通过 time.sleep()方法等待舵机运动完成，为了确保舵机完全到达目标位置，额外增加了 100ms 的延迟。然后，将舵机位置设置为 1000，再次调用 Board.setBusServoPulse()方法来控制舵机运动到新的位置。

2.4 案例 4 调节总线舵机速度

2.4.1 运行程序

（1）打开终端，输入切换到程序所在目录的指令：

“**cd Raspberry_BusServo_demo/**”，按下回车。

```
ubuntu@ubuntu:~/Desktop$ cd Raspberry_BusServo_demo/
ubuntu@ubuntu:~/Desktop/Raspberry_BusServo_demo$
```

（2）输入执行本案例程序的指令：

“**sudo python3 BusServo_Speed.py**”。 舵机以不同的速度转动

```
ubuntu@ubuntu:~/Desktop/Raspberry_BusServo_demo$ sudo python3 BusServo_Speed.py
```

2.4.2 实现效果

程序运行后舵机现象如下：

① 舵机从位置 500 开始；

② 以 0.5 秒的时间转动到位置 1000；

③ 以 1.5 秒的时间转动到位置 500；

④ 以 2.5 秒的时间转动到位置 0；

⑤ 以 3.5 秒的时间转动到位置 500。

2.4.3 案例程序简要分析

- **导入必要的模块**

```
import time
import Board
```

首先，导入 Board 和 time 模块。通过导入该模块，我们可以使用其中定义的函数和类来控制舵机。

- **获取舵机 ID**

```
servo_id = Board.getBusServoID()
```

通过 Board.getBusServoID()函数获取舵机的 ID 信息，并赋值给 servo_id。

- **控制舵机速度**

```
while True:
    # 参数：参数 1：舵机 id；参数 2：位置；参数 3：运行时间
    # 舵机的转动范围 0-240 度，对应的脉宽为 0-1000,即参数 2 的范围为 0-1000

    Board.setBusServoPulse(servo_id, 500, 1000)
    time.sleep(2)
    Board.setBusServoPulse(servo_id, 1000, 500)
    time.sleep(1)
    Board.setBusServoPulse(servo_id, 500, 1500)
    time.sleep(2)
    Board.setBusServoPulse(servo_id, 0, 2500)
    time.sleep(3)
```

```
        Board.setBusServoPulse(servo_id, 500, 3500)
        time.sleep(4)
time.sleep(duration/1000.0 + 0.1)
```

首先，将让舵机回到 500 位置。运行时间设置为 1000ms，然后调用 Board.getBusServoPluse()方法来控制舵机运动到指定位置。

接着，通过 time.sleep()方法等待舵机运动完成。然后，将舵机位置设置为 1000，再次调用 Board.getBusServoPluse()方法来控制舵机运动到新的位置。同样地，通过 time.sleep()方法等待舵机运动完成。接下来，将舵机位置设置为 500，再次调用 Board.getBusServoPluse()方法来控制舵机运动到新的位置。

最后，将舵机位置设置为 0，再次调用 Board.getBusServoPluse()方法来控制舵机运动到新的位置。在每次舵机运动之后，都通过 time.sleep()方法等待舵机运动完成。

- **设置中断标志位**

```
        except KeyboardInterrupt:
            break
```

最后设置按键，当终端界面打印时，按下 Ctrl+C，捕获 KeyboardInterrupt 异常，跳出循环，结束程序的执行。

2.5 案例 5 示教记录操作

2.5.1 运行程序

（1）打开终端，输入切换到程序所在目录的指令：

“**cd Raspberry_BusServo_demo/**”，按下回车。

```
ubuntu@ubuntu:~/Desktop$ cd Raspberry_BusServo_demo/
ubuntu@ubuntu:~/Desktop/Raspberry_BusServo_demo$
```

（2）输入执行本案例程序的指令：

“**sudo python3 BusServo_Record.py**”。 舵机以不同的速度转动

```
ubuntu@ubuntu:~/Desktop/Raspberry_BusServo_demo$ sudo python3 BusServo_Record.py
```

2.5.2 实现效果

程序运行后，回到中位，舵机掉电，提示：可以开始转动舵机，此时掰动舵盘至任意角度，停止后记录当前位置。按下按键 0，此时舵机上电，舵机先回到中位，然后转动按照记录好的角度。pos 为记录好的角度。

```
id: 1
pos1: 500
可以开始转动舵机
input mode:0
pos: 956
```

2.5.3 案例程序简要分析

- **导入必要的模块**

```
import time
import Board
```

首先，导入 Board 和 time 模块。通过导入该模块，我们可以使用其中定义的函数和类来控制舵机。

- **获取舵机 ID 并打印**

```
servo_id = Board.getBusServoID()
print('id:',servo_id)
```

调用 Board.getBusServoID()获取舵机的 ID，并将其打印出来。

- **控制舵机回到中位**

```
Board.setBusServoPulse(servo_id, 500, 1000) # 1 号舵机转到 500 位置用时 1000ms
time.sleep(1)
```

调用 Board.setBusServoPulse()将舵机移动到位置 500 回中，并设置运行时间为

1000ms。

- **获取并打印当前位置**

```
pos1 = Board.setBusServoPulse(servo_id, 500, 1000)
print('pos1:',pos1)
```

调用 Board.setBusServoPulse（）函数获取舵机位置并赋值给 pos1。

- **舵机掉电设置**

```
Board.unloadBusServo(servo_id)
print('可以开始转动舵机')
time.sleep(4)
```

调用 Board.unloadBusServo()将舵机掉电，并打印出“可以开始转动舵机”的提示信息，此时可以用手掰舵机，用 time.sleep()延时 4 秒钟。

- **获取输入模式**

```
mode = int(input('input mode:'))
```

使用 input()函数获取用户输入的模式，并将其转换为整数型。

- **根据获取的模式来执行操作**

```
if mode == 0:
    Board.getBusServoPulse(servo_id)
    pos = Board.getBusServoPulse(servo_id)
    print('pos:',pos)
    Board.setBusServoPulse(servo_id, 0, 1000)
    time.sleep(3)
    Board.setBusServoPulse(servo_id, pos, 1000)
    time.sleep(3)
```

如果模式为 0，首先调用 get_servo_position()获取舵机当前位置，然后将舵机位置设置为 0，并等待 3 秒，最后将舵机位置恢复为之前的位置。